

**LAPORAN TUGAS BESAR MATERI KULIAH PEMROGRAMAN
BERBASIS OBJEK
TEKNIK KOMPUTER
UNIVERSITAS TELKOM**

TOPIK 3



KELOMPOK 12 (TK-46-04):

- **Dwi Agus Cahyo Ilahi (1103223043)**
 - **Ariski Myardi (1103220019)**
- **Rayhan Akbar Al Hafizh (1103223109)**

Dosen Pengampu: Astri Novianty S.T., M.T.

Laporan Aplikasi Perbankan

I. Deskripsi Umum Aplikasi

Fungsionalitas Aplikasi

Aplikasi perbankan ini dirancang untuk memenuhi kebutuhan berbagai peran di dalam bank yang mencakup Admin, Direktur, Customer Service, dan Teller.. Berikut adalah fungsionalitas yang berhasil diimplementasikan untuk setiap kategori pengguna:

Admin:

- Login ke dalam aplikasi
- Menambah/mengedit data direktur
- Menambah/mengedit data teller

Direktur:

- Melihat data transaksi keuangan nasabah
- Melihat laporan keuangan bank
- Melihat data customer service
- Melihat data teller

Customer Service:

- Manambah/mengedit data nasabah

Teller:

- Menambah transaksi nasabah (debit/kredit/bunga/biaya administrasi)
- Melihat data transaksi nasabah

Evaluasi dan Catatan

- Aplikasi telah berhasil mengimplementasikan fungsionalitas dasar untuk setiap kategori pengguna.

II. Rancangan OOP

Jumlah dan Deskripsi Kelas

1. **Admin:** Representasi pengguna dengan peran direktur atau teller.
2. **Direktur:** Pengguna dengan peran direktur memiliki akses untuk melihat laporan keuangan, transaksi nasabah, melihat data customer service, dan data teller.
3. **Customer Service:** Pengguna dengan peran customer service dapat menambah dan mengedit data nasabah.

4. **Teller:** Pengguna dengan peran teller dapat menambah transaksi nasabah dan melihat transaksi nasabah.

Penggunaan Inheritance, Polimorfisme, dan Encapsulation

- **Inheritance:** Kelas “Direktur”, “Customer Service”, dan “Teller” mewarisi sifat umum dari kelas “Admin”.
- **Polimorfisme:** Fungsi “lihat_transaksi_nasabah” dapat dipanggil oleh “Direktur” dan “Teller” dengan implementasi yang berbeda.
- **Encapsulation:** password disimpan secara protected dalam kelas “Admin”

III. Listing Program

```
import hashlib

class Admin:
    def __init__(self, username, password):
        self.username = username
        self.password_hash = self._hash_password(password)

        self.data_direktur = []
        self.data_teller = []
        self.data_nasabah = [
            {
                'nama': 'John Doe',
                'alamat': 'Jl. Contoh No. 123',
                'transaksi': [
                    {"tanggal": "2023-01-15", "jenis": "Transfer Masuk", "jumlah": 5000000, "keterangan": "Dari Rek. 789"},
                    {"tanggal": "2023-01-20", "jenis": "Penarikan Tunai", "jumlah": 2000000, "keterangan": "ATM Lokal"},
                    {"tanggal": "2023-02-02", "jenis": "Pembayaran Tagihan", "jumlah": 1500000, "keterangan": "Listrik"},
                ],
            },
            {
                'nama': 'Jane Smith',
                'alamat': 'Jl. Test No. 456',
                'transaksi': [
                    {"tanggal": "2023-03-10", "jenis": "Transfer Keluar", "jumlah": 3000000, "keterangan": "Ke Rek. 987"},
                    {"tanggal": "2023-03-15", "jenis": "Penarikan Tunai", "jumlah": 1000000, "keterangan": "ATM Lokal"},
                ],
            },
        ],
    },
    },
```

```

# Tambahkan data nasabah lain jika diperlukan
]

def _hash_password(self, password):
    return hashlib.sha256(password.encode()).hexdigest()

def login(self):
    maksimal_percobaan = 3
    percobaan = 0

    while percobaan < maksimal_percobaan:
        input_username = input("Username: ")
        input_password = input("Password: ")

        if input_username == self.username and
self._hash_password(input_password) == self.password_hash:
            print("Login berhasil.")
            return True

        else:
            print(f"Login gagal. Sisa percobaan: {maksimal_percobaan -
percobaan - 1}")
            percobaan += 1

    print("Batas percobaan login tercapai. Aplikasi keluar.")
    exit()

def tambah_edit_direktur(self):
    print("1. Tambah Direktur")
    print("2. Edit Direktur")
    pilihan = int(input("Pilih opsi: "))

    if pilihan == 1:
        nama_direktur = input("Masukkan nama direktur: ")
        jabatan_direktur = input("Masukkan jabatan direktur: ")
        self.data_direktur.append({"nama": nama_direktur, "jabatan":
jabatan_direktur})
        print("Direktur berhasil ditambahkan.")
    elif pilihan == 2:
        print("Data Direktur:")
        for i, direktur in enumerate(self.data_direktur, start=1):
            print(f"{i}. {direktur['nama']} - {direktur['jabatan']}")

        pilihan_edit = int(input("Pilih direktur yang akan diedit (nomor):
"))

        if 1 <= pilihan_edit <= len(self.data_direktur):
            nama_direktur_baru = input("Masukkan nama direktur baru: ")

```

```

        jabatan_direktur_baru = input("Masukkan jabatan direktur baru: ")
    ")
        self.data_direktur[pilihan_edit - 1] = {"nama":
nama_direktur_baru, "jabatan": jabatan_direktur_baru}
        print("Data Direktur berhasil diubah.")
    else:
        print("Nomor direktur tidak valid.")
else:
    print("Opsi tidak valid.")

def tambah_edit_teller(self):
    print("1. Tambah Teller")
    print("2. Edit Teller")
    pilihan = int(input("Pilih opsi: "))

    if pilihan == 1:
        nama_teller = input("Masukkan nama Teller: ")
        cabang_teller = input("Masukkan cabang Teller: ")
        self.data_teller.append({"nama": nama_teller, "cabang":
cabang_teller})
        print("Teller berhasil ditambahkan.")
    elif pilihan == 2:
        print("Data Teller:")
        for i, teller in enumerate(self.data_teller, start=1):
            print(f"{i}. {teller['nama']} - {teller['cabang']}")

        pilihan_edit = int(input("Pilih Teller yang akan diedit (nomor):
"))

        if 1 <= pilihan_edit <= len(self.data_teller):
            nama_teller_baru = input("Masukkan nama Teller baru: ")
            cabang_teller_baru = input("Masukkan cabang Teller baru: ")
            self.data_teller[pilihan_edit - 1] = {"nama":
nama_teller_baru, "cabang": cabang_teller_baru}
            print("Data Teller berhasil diubah.")
        else:
            print("Nomor Teller tidak valid.")
    else:
        print("Opsi tidak valid.")

def save_data_direktur(self):
    with open("data_direktur.txt", "w") as file:
        for direktur in self.data_direktur:
            file.write(f"{direktur['nama']}|{direktur['jabatan']}\n")

def save_data_teller(self):
    with open("data_teller.txt", "w") as file:
        for teller in self.data_teller:
            file.write(f"{teller['nama']}|{teller['cabang']}\n")

```

```

def load_data_direktur(self):
    try:
        with open("data_direktur.txt", "r") as file:
            lines = file.readlines()
            self.data_direktur = [{"nama": line.split('|')[0], "jabatan":
line.split('|')[1].strip()} for line in lines]
            print("Data direktur berhasil dimuat.")
    except FileNotFoundError:
        print("Data direktur tidak ditemukan.")
        self.data_direktur = []

def load_data_teller(self):
    try:
        with open("data_teller.txt", "r") as file:
            lines = file.readlines()
            self.data_teller = [{"nama": line.split('|')[0], "cabang":
line.split('|')[1].strip()} for line in lines]
            print("Data teller berhasil dimuat.")
    except FileNotFoundError:
        print("Data teller tidak ditemukan.")
        self.data_teller = []

def save_laporan_keuangan(self):
    with open("laporan_keuangan.txt", "w") as file:
        for key, value in self.laporan_keuangan.items():
            file.write(f"{key}:{value}\n")

def load_laporan_keuangan(self):
    try:
        with open("laporan_keuangan.txt", "r") as file:
            lines = file.readlines()
            self.laporan_keuangan = {line.split(':')[0].strip():
int(line.split(':')[1].strip()) for line in lines}
            print("Laporan keuangan berhasil dimuat.")
            return self.laporan_keuangan
    except FileNotFoundError:
        print("File laporan keuangan tidak ditemukan. Membuat file baru.")
        self.save_laporan_keuangan({
            'pendapatan': 0,
            'biaya_operasional': 0,
            'laba_bersih': 0
        })
        return {
            'pendapatan': 0,
            'biaya_operasional': 0,
            'laba_bersih': 0
        }

```

```

class Direktur(Admin):
    def __init__(self, username, password):
        super().__init__(username, password)
        self.data_direktur = [
            {"nama": "John Doe", "jabatan": "Direktur Utama"},
            {"nama": "Jane Smith", "jabatan": "Direktur Keuangan"},
        ]
        self.laporan_keuangan = {}
        self.load_data_teller()

    def lihat_transaksi_nasabah(self, id_nasabah):
        # Sebagai contoh, kita akan membuat data transaksi palsu untuk nasabah
        # dengan id tertentu
        transaksi_nasabah = {
            123: [
                {"tanggal": "2023-01-15", "jenis": "Transfer Masuk", "jumlah":
5000000, "keterangan": "Dari Rek. 789"},
                {"tanggal": "2023-01-20", "jenis": "Penarikan Tunai",
"jumlah": 2000000, "keterangan": "ATM Lokal"},
                {"tanggal": "2023-02-02", "jenis": "Pembayaran Tagihan",
"jumlah": 1500000, "keterangan": "Listrik"},
            ],
            # Tambahkan data transaksi untuk nasabah lain jika diperlukan
        }

        if id_nasabah in transaksi_nasabah:
            transaksi_nasabah_nasabah = transaksi_nasabah[id_nasabah]
            print("{:<15} {:<20} {:<15} {:<20}".format("Tanggal", "Jenis
Transaksi", "Jumlah (IDR)", "Keterangan"))
            print("-" * 70)
            for transaksi in transaksi_nasabah_nasabah:
                print("{:<15} {:<20} {:<15} {:<20}".format(
                    transaksi['tanggal'],
                    transaksi['jenis'],
                    format(transaksi['jumlah'], ',d'),
                    transaksi['keterangan']
                ))
            print("-" * 70)
        else:
            print(f"Tidak ada data transaksi untuk nasabah dengan ID
{id_nasabah}")

    def lihat_laporan_keuangan(self):
        print("Laporan Keuangan Bank")
        print("-" * 30)

```

```

        print("Pendapatan          : IDR",
format(self.laporan_keuangan['pendapatan'], ',d'))
        print("Biaya Operasional    : IDR",
format(self.laporan_keuangan['biaya_operasional'], ',d'))
        print("Laba Bersih          : IDR",
format(self.laporan_keuangan['laba_bersih'], ',d'))
        print("-" * 30)

    def lihat_data_cs(self):
        # Sebagai contoh, kita akan membuat data customer service palsu
        data_cs = [
            {"nama": "CS1", "jabatan": "Customer Service", "cabang": "Cabang
A"},
            {"nama": "CS2", "jabatan": "Customer Service", "cabang": "Cabang
B"},
        ]

        print("Data Customer Service")
        print("{:<15} {:<20} {:<15}".format("Nama", "Jabatan", "Cabang"))
        print("-" * 50)
        for cs in data_cs:
            print("{:<15} {:<20} {:<15}".format(cs['nama'], cs['jabatan'],
cs['cabang']))
            print("-" * 50)

    def save_data_teller(self):
        with open("data_teller.txt", "w") as file:
            for teller in self.data_teller:
                file.write(f"{teller['nama']}|{teller.get('cabang', 'Cabang
tidak tersedia')}\n")

    def load_data_teller(self):
        try:
            with open("data_teller.txt", "r") as file:
                lines = file.readlines()
                self.data_teller = [{"nama": line.split('|')[0], "cabang":
line.split('|')[1].strip()} for line in lines]
            print("Data teller berhasil dimuat.")
        except FileNotFoundError:
            print("Data teller tidak ditemukan.")
            self.data_teller = []

    def lihat_data_teller(self):
        print("Data Teller:")
        for i, teller in enumerate(self.data_teller, start=1):
            print(f"{i}. {teller['nama']} - {teller.get('cabang', 'Cabang
tidak tersedia')}")

```

```

def save_laporan_keuangan(self, laporan_keuangan):
    with open("laporan_keuangan.txt", "w") as file:
        for key, value in laporan_keuangan.items():
            file.write(f"{key}:{value}\n")

def load_laporan_keuangan(self):
    try:
        with open("laporan_keuangan.txt", "r") as file:
            lines = file.readlines()
            laporan_keuangan = {line.split(':')[0].strip():
int(line.split(':')[1].strip()) for line in lines}
            print("Laporan keuangan berhasil dimuat.")
            return laporan_keuangan
    except FileNotFoundError:
        print("File laporan keuangan tidak ditemukan. Membuat file baru.")
        self.save_laporan_keuangan({
            'pendapatan': 0,
            'biaya_operasional': 0,
            'laba_bersih': 0
        })
        return {
            'pendapatan': 0,
            'biaya_operasional': 0,
            'laba_bersih': 0
        }

class CustomerService(Admin):
    def tambah_edit_nasabah(self):
        print("1. Tambah Nasabah")
        print("2. Edit Nasabah")
        pilihan = int(input("Pilih opsi: "))

        if pilihan == 1:
            nama_nasabah = input("Masukkan nama nasabah: ")
            alamat_nasabah = input("Masukkan alamat nasabah: ")
            self.data_nasabah.append({"nama": nama_nasabah, "alamat":
alamat_nasabah})
            print("Nasabah berhasil ditambahkan.")
        elif pilihan == 2:
            print("Data Nasabah:")
            for i, nasabah in enumerate(self.data_nasabah, start=1):
                print(f"{i}. {nasabah['nama']} - {nasabah['alamat']}")

            pilihan_edit = int(input("Pilih nasabah yang akan diedit (nomor):
"))

            if 1 <= pilihan_edit <= len(self.data_nasabah):
                nama_nasabah_baru = input("Masukkan nama nasabah baru: ")
                alamat_nasabah_baru = input("Masukkan alamat nasabah baru: ")

```

```

        self.data_nasabah[pilihan_edit - 1] = {"nama":
nama_nasabah_baru, "alamat": alamat_nasabah_baru}
        print("Data Nasabah berhasil diubah.")
    else:
        print("Nomor nasabah tidak valid.")
else:
    print("Opsi tidak valid.")

class Teller(Admin):
    def input_pilihan_transaksi(self):
        print("1. Debit")
        print("2. Kredit")
        print("3. Bunga")
        print("4. Biaya Administrasi")
        return int(input("Pilih jenis transaksi: "))

    def input_pilihan_nasabah(self):
        print("Data Nasabah:")
        for i, nasabah in enumerate(self.data_nasabah, start=1):
            print(f"{i}. {nasabah['nama']} - {nasabah['alamat']} - Saldo:
{nasabah.get('saldo', 0)}")
        return int(input("Pilih nasabah (nomor): "))

    def input_jumlah_transaksi(self):
        return int(input("Masukkan jumlah transaksi: "))

    def tambah_transaksi_nasabah(self):
        pilihan = self.input_pilihan_transaksi()
        pilihan_nasabah = self.input_pilihan_nasabah()

        if 1 <= pilihan_nasabah <= len(self.data_nasabah):
            jumlah_transaksi = self.input_jumlah_transaksi()

            if 'saldo' not in self.data_nasabah[pilihan_nasabah - 1]:
                self.data_nasabah[pilihan_nasabah - 1]['saldo'] = 0

            self._proses_transaksi(pilihan, pilihan_nasabah, jumlah_transaksi)
        else:
            print("Nomor nasabah tidak valid.")

    def lihat_transaksi_nasabah(self):
        pilihan_nasabah = self.input_pilihan_nasabah()

        if 1 <= pilihan_nasabah <= len(self.data_nasabah):
            transaksi_nasabah = self.data_nasabah[pilihan_nasabah -
1].get('transaksi', [])

```

```

        print("{:<15} {:<20} {:<15} {:<20}".format("Tanggal", "Jenis
Transaksi", "Jumlah (IDR)", "Keterangan"))
        print("-" * 70)

        if transaksi_nasabah:
            for transaksi in transaksi_nasabah:
                print("{:<15} {:<20} {:<15} {:<20}".format(
                    transaksi['tanggal'],
                    transaksi['jenis'],
                    format(transaksi['jumlah'], ',d'),
                    transaksi['keterangan']
                ))
            else:
                print("Tidak ada data transaksi untuk nasabah ini.")

        print("-" * 70)
    else:
        print("Nomor nasabah tidak valid.")

    def _proses_transaksi(self, pilihan, pilihan_nasabah, jumlah_transaksi):
        if pilihan == 1:
            # Debit
            self.data_nasabah[pilihan_nasabah - 1]['saldo'] -=
jumlah_transaksi
            print("Transaksi Debit berhasil.")
        elif pilihan == 2:
            # Kredit
            self.data_nasabah[pilihan_nasabah - 1]['saldo'] +=
jumlah_transaksi
            print("Transaksi Kredit berhasil.")
        elif pilihan == 3:
            # Bunga
            bunga = 0.05 # Misalnya, bunga sebesar 5%
            self.data_nasabah[pilihan_nasabah - 1]['saldo'] +=
int(jumlah_transaksi * (1 + bunga))
            print("Transaksi Bunga berhasil.")
        elif pilihan == 4:
            # Biaya Administrasi
            biaya_administrasi = 10000 # Misalnya, biaya administrasi sebesar
10,000
            self.data_nasabah[pilihan_nasabah - 1]['saldo'] -=
biaya_administrasi
            print("Transaksi Biaya Administrasi berhasil.")
        else:
            print("Jenis transaksi tidak valid.")

```

```

# Inisialisasi admin dan direktur dan teller dan cs
admin = Admin("admin", "password123")
direktur = Direktur("direktur", "password456")
teller = Teller("teller", "password123")
cs = CustomerService("cs", "password123")
# Muat data dari file saat aplikasi dimulai
admin.load_data_direktur()
direktur.laporan_keuangan = admin.load_laporan_keuangan()

admin.data_direktur = direktur.data_direktur

while True:
    print("Selamat Datang Di Aplikasi Perbankan")
    pilihan = int(input('''1. Admin
2. Direktur
3. Customer Service
4. Teller
5. Quit

Input: '''))

    if pilihan == 1:
        if admin.login():
            print("1. Menambah/Mengedit data direktur")
            print("2. Menambah/Mengedit data teller", end="")
            pilihan = int(input("\nPilih opsi: "))
            if pilihan == 1:
                # Menambah/Mengedit data direktur
                admin.tambah_edit_direktur()
                # Simpan perubahan ke file
                admin.save_data_direktur()
            elif pilihan == 2:
                # Menambah/Mengedit data teller
                admin.tambah_edit_teller()
                # Simpan perubahan ke file
                admin.save_data_teller()

        elif pilihan == 2:
            pilihanDirektur = int(input('''1. Melihat data transaksi keuangan
nasabah
2. Melihat laporan keuangan bank
3. Melihat data customer service
4. Melihat data teller
input: '''))

            if pilihanDirektur == 1:

```

```

        direktur.lihat_transaksi_nasabah(123)
    elif pilihanDirektur == 2:
        direktur.lihat_laporan_keuangan()
    elif pilihanDirektur == 3:
        direktur.lihat_data_cs()
    elif pilihanDirektur == 4:
        direktur.lihat_data_teller()

elif pilihan == 3:
    print("Menambah/Mengedit data nasabah")
    cs.tambah_edit_nasabah()

elif pilihan == 4:
    pilihanTeller = int(input('''1. Menambah transaksi nasabah
(debit/kredit/bunga/biaya administrasi)
2. Melihat data transaksi nasabah
input: '''))
    if pilihanTeller == 1:
        teller.tambah_transaksi_nasabah()
    elif pilihanTeller == 2:
        teller.lihat_transaksi_nasabah()
elif pilihan == 5:
    # Simpan data direktur dan laporan keuangan ke file sebelum keluar
    admin.save_data_direktur()
    admin.save_laporan_keuangan(direktur.laporan_keuangan)
    print('Terima Kasih telah menggunakan Program ini...')
    break

```